TECHNICAL PROJECT REPORT

Slate TimeSync

Human Resource Management System (HRMS)

Workforce Time, Attendance, Leave and Role-Based Access Platform

Spring Boot 3 | React 18 | PostgreSQL 16 | Redis 7 | Kafka | Docker

Project	Slate TimeSync - HRMS (slate-timesync)
Report Title	Full Project Analysis and Technical Documentation
Backend Version	1.0.0 (hrms-backend, Maven artifact)
Document Version	1.0
Report Date	September 4, 2026
Architecture	Modular Monolith REST API + React SPA + Event-Driven Notifications
Prepared By	Engineering Analysis (automated codebase review)

Document Scope

This report is based entirely on the actual source code in this workspace. Every architectural claim, configuration value, endpoint and workflow described here was verified by direct inspection of the code. Verification results (build, lint, test inventory) reflect real commands executed during report preparation; checks that could not be executed in the authoring environment are explicitly marked as such.

Table of Contents

1	Executive Summary	3
2	Project Overview	4
3	Functional Requirements	6
4	Non-Functional Requirements	8
5	Features and Functionalities	9
6	Technology Stack	11
7	System Architecture	13
8	Detailed Component Architecture	14
9	Frontend Architecture	15
10	Backend Architecture	16
11	API Documentation	17
12	Database Architecture	19
13	Authentication and Authorization	21
14	Complete Application Workflows	22
15	Complete Data Flow	23
16	Project Folder Structure	24
17	Module and Implementation Details	25
18	Validation and Error Handling	26
19	Security Analysis	27
20	Testing	28
21	Verification and Quality Assurance	29
22	Performance Analysis	30
23	Scalability	31
24	Deployment and DevOps	32
25	Installation and Setup	33
26	Problems Found and Improvements	34
27	Current Strengths	35
28	Known Limitations	36
29	Recommended Improvements	37
30	Future Enhancements	38
31	Maintenance and Support	39
32	GitHub / Repository Readiness	40
33	Resume / Portfolio Value	41
34	Final Technical Summary	42
35	Appendix	43

1 Executive Summary

Slate TimeSync is a full-stack Human Resource Management System (HRMS) that digitises the daily operational core of workforce management: employee records, attendance (clock in/out, breaks, corrections, holidays, policies), leave management (types, balances, approvals) and weekly timesheets (entries, submission, approval), wrapped in a dynamic role-based access control (RBAC) engine, real-time notifications, and an auditable event pipeline.

The system solves a concrete operational problem: organisations still tracking attendance on spreadsheets and approving leave over email have no single, permission-aware source of truth. Slate TimeSync consolidates these workflows behind one authenticated web application with clear separation between employee, manager, HR and super-admin capabilities.

Target users. Employees (self-service attendance, timesheets, leave), Managers (team visibility and approvals), HR administrators (directory, policies, balances, corrections) and Super Administrators (roles, permissions, audit, system configuration).

Technical approach. A Java 21 / Spring Boot 3.3 modular monolith exposes a versioned REST API (/api/v1) backed by PostgreSQL 16 (Flyway-managed schema of 22 tables), Redis 7 (caching, rate limiting, event idempotency) and Apache Kafka (asynchronous notification delivery via a transactional outbox). The frontend is a React 18 + TypeScript single-page application built with Vite, styled with Tailwind CSS and shadcn/ui, using TanStack Query for server state and WebSocket for live notifications. Docker Compose provisions the entire stack, including Prometheus and Grafana observability.

Project origin. The frontend was originally generated on the Lovable platform against a Supabase backend. The codebase documents a deliberate migration to a self-hosted Spring Boot stack: the Supabase migrations remain under supabase/migrations as schema history, the API client comment states it 'replaces the Supabase integration', and RbacService explicitly documents that it 'replaces the Supabase RLS policies'.

Implementation status. The functional core is complete and internally consistent: 50+ REST endpoints across 8 controllers, transactional business services, JWT authentication with refresh-token rotation, OAuth2 (Google, GitHub), an event-driven notification pipeline with email delivery, audit logging, and a pre-provisioned monitoring stack. During report preparation the frontend production build succeeded; the backend test suite (10 tests in 3 classes) exists but could not be executed in the authoring environment (no Java/Maven/Docker installed), as reported honestly in Sections 20-22.

2 Project Overview

2.1 Purpose and Problem Statement

Manual HR operations produce three recurring failures: (1) attendance data that cannot be trusted because it is self-reported and unvalidated; (2) leave approvals that lack balance enforcement and an audit trail; (3) timesheet approval that depends on email chains with no central status. Slate TimeSync addresses all three with server-enforced rules (unique daily attendance, overlap and balance checks on leave, state-machine transitions on timesheets), a permission model enforced at every API boundary, and an append-only audit log for administrative actions.

2.2 Objectives

- Provide authenticated, role-aware access to all HR workflows with email verification and account protection (lockout, rate limiting).
- Automate attendance capture with policy-driven lateness, break tracking, and manager-approved corrections.
- Enforce leave governance: configurable types, yearly balances, overlap prevention, and approve/reject decisions with mandatory rejection reasons.
- Support weekly timesheet cycles (draft, submit, approve/reject, resubmit) with bulk approval for approvers.
- Deliver notifications reliably: in-app (persisted + WebSocket push) and email (SMTP via Kafka, retried, dead-lettered on failure).
- Make every privileged action auditable and observable (audit logs, correlation IDs, Prometheus metrics, Grafana dashboards).

2.3 Scope

In Scope (implemented)	Out of Scope (not identified in the current implementation)
Authentication: local + Google/GitHub OAuth2, email verification, password reset, refresh-token rotation	Payroll processing and salary structures
Attendance: clock in/out, breaks, corrections, holidays, policies, monthly report	Shift/roster scheduling and shift-swap workflows
Leave: types, balances, requests, approvals, cancellation	Recruitment, onboarding pipelines, applicant tracking
Timesheets: weekly sheets, project/task entries, billable flags, approvals	Expense management and reimbursements
Employees: directory, search/pagination, create/update, onboarding/offboarding flag	Performance reviews / goal-setting cycles (reports cover utilisation only)
RBAC: 4 seeded roles, 20 permissions, custom roles, assignments, permission caching	Multi-tenancy (single organisation per deployment)
Notifications: in-app + WebSocket + transactional email; reminders (Supabase functions)	Mobile native applications; SMS/push channels
Observability: Prometheus metrics, Grafana dashboard, correlation IDs, rolling logs	Distributed tracing (OpenTelemetry) - not configured

2.4 Use Cases (derived from implemented endpoints and pages)

#	Actor	Use Case	Primary Components
1	Employee	Sign up, verify email via SMTP link, sign in	AuthController, EmailService, VerifyEmail page
2	Employee	Clock in / start break / end break / clock out	AttendanceController/Service, Attendance page
3	Employee	Submit weekly timesheet with project entries	TimesheetController/Service, Timesheets page
4	Employee	Apply for leave within remaining balance	LeaveController/Service, Leave page
5	Manager	Approve/reject team leave and timesheets (incl. bulk)	Leave/Timesheet decision endpoints
6	Manager/HR	Approve attendance corrections with amended times	Attendance corrections endpoints
7	HR	Manage employee directory, onboard/offboard profiles	EmployeeController, TeamManagement page
8	HR	Configure leave balances and attendance policies	Leave balances endpoint, Attendance policy endpoints

#	Actor	Use Case	Primary Components
9	Super Admin	Create roles, grant permissions, assign roles	RbacController, RoleManagement page
10	Super Admin	Review audit trail of privileged changes	AuditController, /api/v1/admin/audit-logs
11	All	Receive real-time in-app and email notifications	Kafka consumer, WebSocketHandler, bell UI
12	Employee	Import calendar events as timesheet entries	CalendarImport component (file upload, mapping)

2.5 Expected Outcomes

- A single permission-aware system of record for attendance, leave and timesheets.
- Managers act on pending approvals instead of chasing email threads; employees see decisions in real time.
- Every privileged change is attributable (audit_logs, rbac_audit_logs, attendance_audit_logs).
- Operations teams get metrics and dashboards out of the box (Prometheus + Grafana provisioning included).

3 Functional Requirements

Requirements below are reverse-engineered from the implemented controllers, services, database constraints and UI pages. 'Verified' means the behaviour is enforced in code (not merely claimed in documentation).

ID	Requirement	Enforced By (evidence)	Status
FR-A1	Register with email/password; duplicate emails rejected (case-insensitive)	AuthService.signUp + users.email UNIQUE + existsByEmailIgnoreCase	**Verified**
FR-A2	Email verification required before first sign-in	SignIn throws EMAIL_NOT_VERIFIED; one_time_tokens purpose=EMAIL_VERIFICATION	**Verified**
FR-A3	Password reset via one-time emailed token; all sessions revoked on reset	forgotPassword / resetPassword; revokeAllSessions(userId)	**Verified**
FR-A4	JWT access (15 min) + refresh token (7 days) with rotation and revocation	JwtTokenProvider; refresh_tokens.token_hash; AuthService.refresh	**Verified**
FR-A5	Account lockout after 5 failed logins for 15 minutes	AuthService MAX_FAILED_ATTEMPTS/LOCK_MINUTES; users.locked_until	**Verified**
FR-A6	OAuth2 sign-in with Google and GitHub; user auto-provisioned	SecurityConfig oauth2Login, CustomOAuth2UserService, OAuth2LoginSuccessHandler	**Verified**
FR-A7	Rate limiting: 20 req/min on auth, 300 req/min on API (per IP)	RateLimitFilter (Redis counters, 429 response)	**Verified**
FR-AT1	One attendance record per employee per day (clock in once)	UNIQUE(profile_id, date); checkIn business-rule guard	**Verified**
FR-AT2	Late status derived from policy grace period	AttendanceService.isLate() vs attendance_policies	**Verified**
FR-AT3	Break start/end windows recorded; totals computed on checkout	startBreak/endBreak; computeTotalHours	**Verified**
FR-AT4	Employees can request corrections; approvers amend times and status	POST /corrections; approveCorrectionExtended	**Verified**
FR-AT5	Holidays manageable by HR; viewable by year (recurring supported)	HolidayRepository, /attendance/holidays	**Verified**
FR-AT6	Monthly attendance report per employee (counts by status)	GET /attendance/report/{profileId}	**Verified**
FR-L1	Leave request validated: active type, date order, no overlap, balance sufficiency	LeaveService.createRequest checks (4 guards)	**Verified**
FR-L2	Approve/reject with mandatory rejection reason; approval stamps user+time	LeaveService.decide	**Verified**
FR-L3	Only pending requests cancellable	LeaveService.cancel guard	**Verified**
FR-L4	Role-aware request visibility (own / team / all)	LeaveService.visibleRequests switch on effectiveRole	**Verified**
FR-L5	Leave types and balances administrable	types CRUD; POST /leaves/balances (ensureBalance)	**Verified**
FR-T1	Weekly timesheet auto-created (Monday-anchored) as draft	TimesheetService.createOrGet	**Verified**
FR-T2	Entries editable only in draft/rejected state; totals recalculated	saveEntry guard; recalculate(ts)	**Verified**
FR-T3	Submit, approve, reject (reason required); bulk approve	submit/decide/bulkApprove	**Verified**
FR-T4	Billable flag and project name per entry	TimesheetEntry fields; SaveEntry record	**Verified**

ID	Requirement	Enforced By (evidence)	Status
FR-E1	Employee search with pagination (max 100 page size) + department filter	EmployeeService.search (ProfileRepository.search)	**Verified**
FR-E2	Create employee with unique email and employee ID (auto EMP-xxxxxxxx)	EmployeeService.create	**Verified**
FR-E3	Update deactivation flag implements onboarding/offboarding	EmployeeService.update (isActive)	**Verified**
FR-R1	Dynamic roles + module/action permissions; 4 system roles and 20 permissions seeded	V2__seed.sql; roles/permissions/role_permissions	**Verified**

ID	Requirement	Enforced By (evidence)	Status
FR-R2	Every privileged endpoint checks requirePermission(userId, module, action)	RbacService; controllers	**Verified**
FR-R3	Permission lookups cached in Redis (5 min TTL) with invalidation on change	RbacService.cache/invalidateUserCache	**Verified**
FR-R4	Legacy 4-value role system combined with RBAC for effective role	RbacService.effectiveRole precedence	**Verified**
FR-N1	In-app notifications persisted and pushed live over WebSocket	NotificationService.push; NotificationWebSocketHandler	**Verified**
FR-N2	Business events produce emails asynchronously with retry and DLT	OutboxService -> Kafka -> HrmsEventConsumer; EmailService retry x3	**Verified**
FR-N3	Duplicate event consumption suppressed (7-day idempotency window)	Redis setIfAbsent on hrms:evdone:{eventId}	**Verified**
FR-P1	Weekly performance metrics per employee (hours, attendance %, utilisation, score)	ReportController.performance	**Verified**
FR-P2	Per-employee drill-down: 8-week hours, 30-day attendance trend, workload split	ReportController.detail	**Verified**
FR-X1	Audit log of privileged actions, paged, admin-only	AuditRecorder; AuditController (role_management:view)	**Verified**
FR-X2	Correlation ID + user ID attached to every log line	CorrelationIdFilter; JwtAuthenticationFilter MDC	**Verified**

4 Non-Functional Requirements

The table distinguishes between properties actively supported by the implementation and properties that are recommendations only. No unverified claim is made.

Dimension	Evidence in Implementation	Assessment
Performance	Hikari pool (max 10); Redis caches (permissions TTL 300 s); DB indexes on hot paths (attendance profile+date, refresh token expiry, outbox unprocessed); rate limiting protects auth endpoints; Vite production build.	Partial; no load testing performed
Scalability	Stateless API (JWT, no server sessions) allows horizontal scaling; Kafka decouples notification load; Redis centralises counters/cache.	Partial; single-node Postgres is a ceiling
Security	BCrypt cost 12; hashed refresh/one-time tokens; lockout; rate limiting; CORS whitelist; RBAC on every privileged endpoint; secrets via environment; include-stacktrace: never.	Strong baseline; see Section 19
Reliability	Transactional outbox with retry; Kafka ErrorHandlingDeserializer + DLT (hrms.events.dlt); email retry (3 attempts, backoff); Redis cache degrades gracefully to DB; compose healthchecks.	Supported by design; failover untested
Maintainability	Package-by-feature layout (com.hrms.<module>); Flyway versioned schema; DTO records per controller; central exception model; Swagger annotations.	Supported
Usability	shadcn/ui component system, dark theme default, toast feedback, role-gated navigation (useRBAC), live notification bell.	Supported (qualitative)
Availability	Compose healthchecks (postgres/redis/kafka) with depends_on conditions; actuator health endpoint exposed.	Partial; no HA configuration
Accessibility	Built on Radix UI primitives (keyboard/ARIA behaviour inherited); no dedicated accessibility audit identified.	**Not verified**
Data Integrity	PK/FK constraints, UNIQUE(profile_id,date), CHECK constraints on enums (status, provider, role), ddl-auto: validate, optimistic-lock version columns, audit snapshots.	Supported
Observability	Actuator (health/info/metrics/prometheus); Micrometer counters (hrms.email.sent/failed, hrms.events.processed/failed); Logback rolling logs (14 days) with MDC; Grafana dashboard provisioned.	Supported

5 Features and Functionalities

5.1 Implemented Features

Feature	What It Does / How It Works	Key Components	Status
Authentication suite	Signup sends a real verification email (via outbox + Kafka). Sign-in validates credentials, verification state, lockout and status, then issues an access JWT (15 min) and a rotating refresh token stored as SHA-256 hash. Logout revokes; forgot/reset/change-password flows included.	AuthService, AuthController, JwtTokenProvider, one_time_tokens, refresh_tokens	Implemented
OAuth2 social login	Spring Security oauth2Login for Google and GitHub; CustomOAuth2UserService provisions the local user; success handler issues the same JWT pair.	SecurityConfig, CustomOAuth2UserService, OAuth2LoginSuccessHandler	Implemented
Rate limiting	Redis sliding counters per IP: 20/min on /api/v1/auth/*, 300/min elsewhere; 429 with JSON error; auth endpoints protected against brute force in combination with lockout.	RateLimitFilter	Implemented
Attendance tracking	Daily clock in/out, break windows, total hours, policy-driven 'late' status, notes; one record per person per day enforced by DB constraint.	AttendanceController/Service, attendance table	Implemented
Attendance corrections	Employee submits correction with reason; approver view lists pending; approval can amend check-in/out times, recompute hours, set status; audit snapshot written; event emitted.	corrections endpoints, attendance_audit_logs, ATTENDANCE_CORRECTED event	Implemented
Holidays & policies	Year-filtered holiday calendar (recurring support); attendance policy CRUD with working hours, grace period, break and overtime threshold; 'late' logic reads the active policy.	holidays, attendance_policies	Implemented
Monthly report	GET /attendance/report/{profileId} aggregates present/late/absent/half-day counts for a year-month.	AttendanceController.monthlyReport	Implemented
Leave management	Type catalogue (4 seeded types), yearly balances per type, requests with overlap + balance guards, approve/reject with mandatory reason, cancel while pending, role-aware listing.	LeaveController/Service, leave_* tables	Implemented
Timesheets	Monday-anchored weekly sheet auto-created as draft; per-entry project/task/hours/billable; totals recalculated on every change; draft/rejected editable only; submit/approve/reject + bulk approve; team visibility by role.	TimesheetController/Service, timesheets, timesheet_entries	Implemented
Calendar import (timesheets)	Upload a calendar export file, map events to projects, select events, then bulk-create timesheet entries for the current week.	components/timesheets/CalendarImport.tsx	Implemented
Employee directory	Paged search (search + department filters), create with unique employee ID (auto-generated EMP- prefix), update, active-flag onboarding/offboarding; /employees/me resolves own profile.	EmployeeController/Service	Implemented
RBAC engine	4 seeded roles x 20 module:action permissions; custom roles; role-permission grants; user assignments; effective role merges legacy user_roles with assignments; Redis-cached permissions (5 min TTL) invalidated on change.	RbacController/Service, roles, permissions, user_role_assignments	Implemented
Performance reports	Weekly per-employee scorecard (hours vs 40h target, attendance rate, utilisation, composite score) plus 8-week hours trend, 30-day attendance trend and workload breakdown.	ReportController	Implemented
Notification pipeline	Transactional outbox -> Kafka -> consumer creates in-app notifications (persisted + WebSocket push) and emails (SendGrid SMTP, 3 retries). Redis idempotency prevents double-processing; failures land in DLT.	OutboxService, HrmsEventConsumer, EmailService, NotificationService	Implemented
WebSocket live updates	Client connects to /ws/notifications with JWT in query; handler pushes per-user JSON payloads; client auto-reconnects every 5 s.	WebSocketConfig, NotificationWebSocketHandler, api.connectNotifications	Implemented
Audit logging	Append-only audit_logs with old/new value snapshots for employees, roles, role assignments; separate rbac_audit_logs and attendance_audit_logs; admin-only paged API.	AuditRecorder, AuditController	Implemented
Observability	Actuator + Micrometer Prometheus endpoint; hrms.* business counters; CorrelationIdFilter propagates X-Correlation-Id; Logback pattern includes correlationId and userId; Grafana dashboard provisioned.	application.yml, logback-spring.xml, docker/grafana	Implemented

Feature	What It Does / How It Works	Key Components	Status
PDF/XLSX exports (client)	Frontend bundles jspdf + jspdf-autotable and xlsx for client-side report/export generation in HR screens.	package.json dependencies	Implemented

5.2 Partially Implemented / Legacy

Item	Observation	Status
Supabase edge functions	Five Deno functions (reminder/notification emails) remain under supabase/functions; they are not part of the Spring Boot runtime and would only run on Supabase infrastructure. Kept as legacy/history.	Legacy
AttendanceReminderJob	Backend scheduled job exists for attendance reminders; its runtime behaviour was reviewed statically only (not executed in this environment).	Implemented (static review)
Team attendance view	GET /attendance/team returns all records in range for non-employees (role string check), without manager-scope filtering.	Partial

6 Technology Stack

Technology	Version	Purpose	Where Used
Java	21	Backend language (virtual-thread era LTS)	backend/pom.xml (java.version)
Spring Boot	3.3.5	Application framework (modular monolith)	backend
Spring Security + OAuth2 Client	6.x (Boot-managed)	JWT stateless auth, BCrypt, Google/GitHub login	SecurityConfig, auth package
Spring Data JPA / Hibernate	Boot-managed	ORM, repositories, ddl-auto validate	All services
PostgreSQL	16-alpine	Primary relational database	docker-compose, Flyway V1/V2
Flyway	10.x (Boot-managed)	Versioned migrations + seed data	V1__init.sql, V2__seed.sql
Redis	7-alpine	Cache (permissions), rate limiting, event idempotency	RedisConfig, RateLimitFilter, RbacService, HrmsEventConsumer
Apache Kafka (bitnami)	3.7	Async event bus for notifications	KafkaConfig, OutboxService, HrmsEventConsumer
JJWT	0.12.6	JWT creation/parsing (HMAC-SHA)	JwtTokenProvider
Spring Kafka	Boot-managed	Producer/consumer, JsonSerializer, DLT	KafkaConfig, KafkaErrorConfig
Spring WebSocket	Boot-managed	Real-time notification push	WebSocketConfig, NotificationWebSocketHandler
Spring Mail (Jakarta Mail)	Boot-managed	SMTP email (SendGrid relay)	EmailService, EmailTemplates
Flyway + Testcontainers + H2	test scope	Integration testing against real Postgres	backend tests
springdoc-openapi	2.6.0	Swagger UI / OpenAPI docs	OpenApiConfig, /swagger-ui.html
Micrometer + Prometheus registry	Boot-managed	Metrics export for Prometheus	Actuator config
Lombok	Boot-managed	Boilerplate reduction	Entities
Logback	Boot-managed	Structured logging, MDC, rolling files	logback-spring.xml
React	18.3	UI library	frontend/src
TypeScript	5.8	Type-safe frontend	All frontend sources
Vite	5.4	Dev server + production bundler (SWC)	vite.config.ts
Tailwind CSS + tailwindcss-animate	3.4	Utility-first styling	tailwind.config.ts
shadcn/ui + Radix primitives	various	Accessible component kit (48 UI components)	src/components/ui
TanStack Query	5.83	Server-state cache/mutations	App QueryClientProvider, hooks
react-router-dom	6.30	Client routing (14 routes)	App.tsx
react-hook-form + zod + @hookform/resolvers	7.6 / 3.25	Form handling and schema validation	Forms in HR screens
recharts	2.15	Dashboard charts (performance)	Performance page
date-fns	3.6	Date handling	Timesheet/calendar components
jspdf + jspdf-autotable + xlsx	3.0 / 5.0 / 0.18	Client-side PDF and Excel export	HR report screens
lucide-react	0.462	Icon set	UI
Docker / Docker Compose	-	Containerised build + full local stack	backend/Dockerfile, frontend/Dockerfile, docker/docker-compose.yml

Technology	Version	Purpose	Where Used
Nginx	1.27-alpine	Static SPA hosting + reverse proxy (/api, /ws)	frontend/nginx.conf
Prometheus / Grafana	latest	Metrics scraping + dashboards	docker/prometheus, docker/grafana
Deno (legacy)	std 0.190	Supabase edge functions (reminder emails)	supabase/functions
ESLint + typescript-eslint	9.32 / 8.38	Static analysis (frontend)	eslint.config.js

7 System Architecture

Slate TimeSync is a modular monolith: one Spring Boot deployment owns all business modules, chosen deliberately to keep transactional integrity (single database, single transaction manager) while retaining module boundaries that could later be extracted into services. Asynchronous work (emails, notifications) is decoupled through Kafka using the transactional outbox pattern, so a Kafka or SMTP outage can never roll back a business transaction.

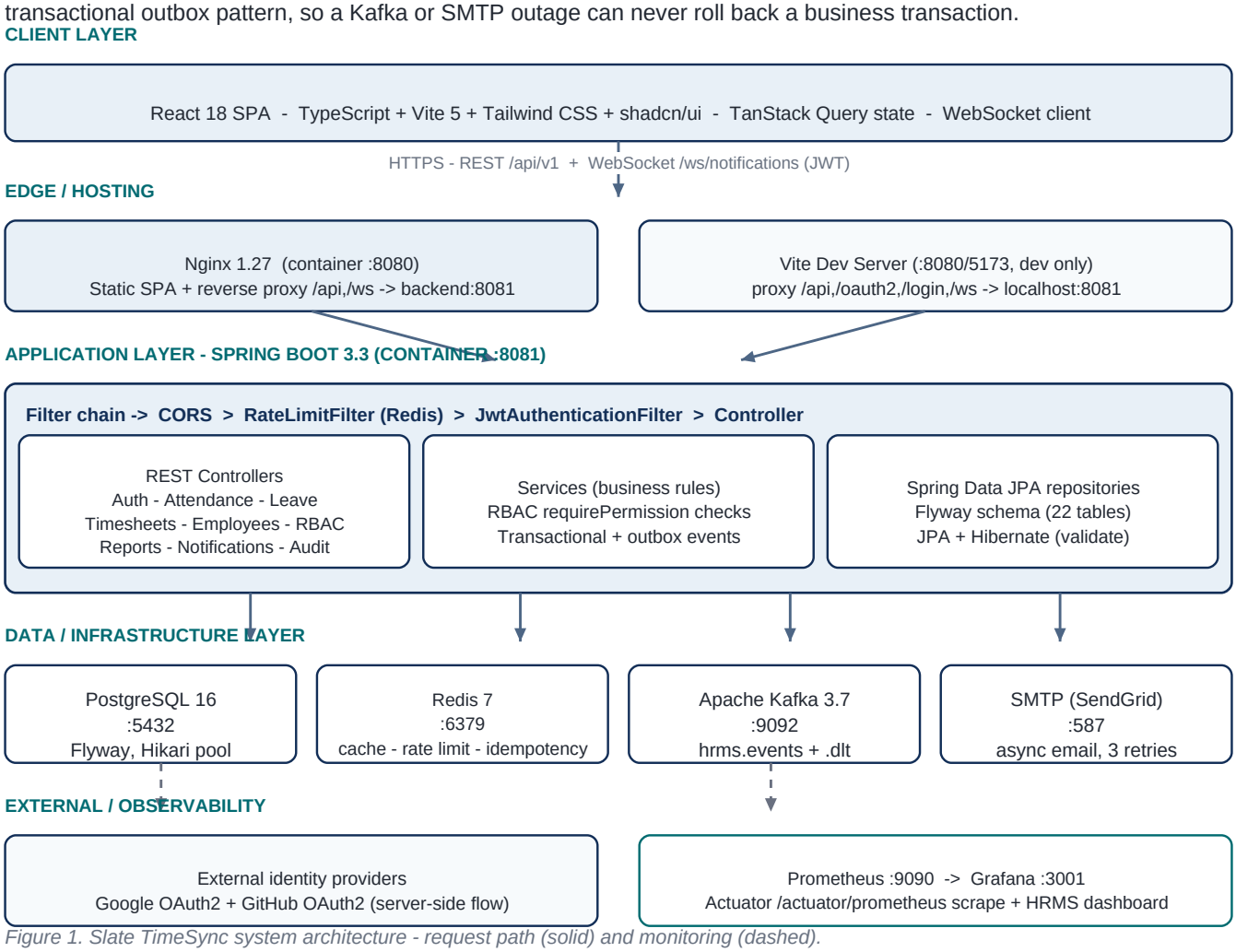


Figure 1. Slate TimeSync system architecture - request path (solid) and monitoring (dashed).

Flow of a typical request: the SPA calls same-origin /api/v1 paths. In production Nginx serves the built bundle and proxies /api and /ws to the backend container; in development the Vite dev server performs the same proxying. The backend applies CORS, Redis-backed rate limiting and JWT validation before any controller executes. Services enforce RBAC permissions, mutate PostgreSQL through JPA, and append outbox events inside the same transaction. A scheduled dispatcher publishes outbox events to Kafka; the notification consumer fans out to in-app messages (WebSocket) and email.

8 Detailed Component Architecture

Component	Responsibility	Processing	Outputs / Interactions
AuthController / AuthService	Sign up, sign in, refresh (rotation), logout, forgot/reset/change password, verify/resend email, /me	Uniqueness check, BCrypt encode/verify, lockout counters, verification gate, token hashing (SHA-256), session revocation	JWT pair; outbox EMAIL_VERIFICATION / PASSWORD_RESET events; users / refresh_tokens / one_time_tokens
JwtTokenProvider	Issue/parse HMAC-signed access tokens	Claims: sub, email, roles, type=access, 15-min expiry	Signed JWT; parsed claims for the filter
JwtAuthenticationFilter	Authenticate every request from Bearer header	Signature + expiry check, build UserPrincipal, set SecurityContext and MDC userId	Authenticated request or anonymous (401 downstream)
RateLimitFilter	Per-IP sliding window limits	Redis INCR with 60 s expiry; auth 20/min, API 300/min	X-RateLimit-Limit header; 429 JSON on breach; skipped for /ws and /actuator
AttendanceController/Service	Daily attendance lifecycle, corrections, holidays, policies, monthly report	Duplicate-day guard, grace-period lateness, total-hours computation, audit snapshots, correction approval	attendance rows, attendance_audit_logs, ATTENDANCE_CORRECTED event
LeaveController/Service	Leave types, balances, requests, decisions	4 validation guards; state transitions; visibility by role	leave_requests rows, LEAVE_* events, leave_balances
TimesheetController/Service	Weekly timesheet lifecycle	Monday anchoring, ownership check, state machine, totals recalculation, bulk approve	timesheets / timesheet_entries, TIMESHEET_* events
EmployeeController/Service	Directory CRUD + paged search	Duplicate email/ID guards, EMP-id generation, audit record	profiles rows, EMPLOYEE_* audit logs
RbacController/Service	Roles, permissions, grants, assignments, effective role	Uniqueness checks, cache invalidation, audit	roles / permissions tables, Redis perm cache, ROLE_* audit logs
ReportController	Performance metrics and drill-down	Aggregation over timesheet sums + attendance counts	JSON metric maps (scorecard, trends, workload)
OutboxService	Transactional outbox	append() inside caller txn; scheduled dispatcher publishes up to 100 pending; retries++ on failure; 7-day purge	Kafka hrms.events; outbox_events table
HrmsEventConsumer	Event -> notifications fan-out	Redis idempotency (7 d), per-type switch, Micrometer counters	notifications rows + WebSocket push + EmailService send; failures -> DLT
EmailService	SMTP delivery	3 attempts with linear backoff; MAIL_ENABLED=false logs instead; Micrometer counters	Email delivered or permanent-failure log
NotificationService + WebSocket	In-app notifications	Persist + JSON push to the user's sessions; read / read-all transitions	notifications table, WebSocket frames
AuditRecorder / AuditController	Privileged-change trail	Append-only insert with old/new snapshots; paged admin query gated by permission	audit_logs rows; /api/v1/admin/audit-logs
RbacService	Authorization core	Effective-role precedence; permission load from assignments; Redis cache (5 min) with DB fallback	hasPermission / requirePermission; 403 via AccessDeniedException
GlobalExceptionHandler / ApiException	Uniform error contract	Map ApiException / AccessDenied / validation errors to HTTP status + {code, message} JSON; no stack traces	ErrorResponse JSON
Frontend api client (lib/api.ts)	All HTTP access	Attach Bearer; on 401 single-flight refresh then one retry; normalise ApiError; 204 passthrough	Parsed JSON or ApiError; tokens in localStorage
useAuth / useRBAC hooks	Session + permission state	GET /auth/me bootstrap; signIn/signUp/signOut wrappers; role gates	AuthState consumed by pages

9 Frontend Architecture

The frontend is a React 18 + TypeScript SPA scaffolded on Vite with the shadcn/ui ecosystem. Server state lives in TanStack Query; ephemeral UI state lives in hooks; there is no global Redux-style store. All HTTP access is funnelled through one typed client (src/lib/api.ts) that owns token storage, transparent refresh-token rotation and error normalisation.

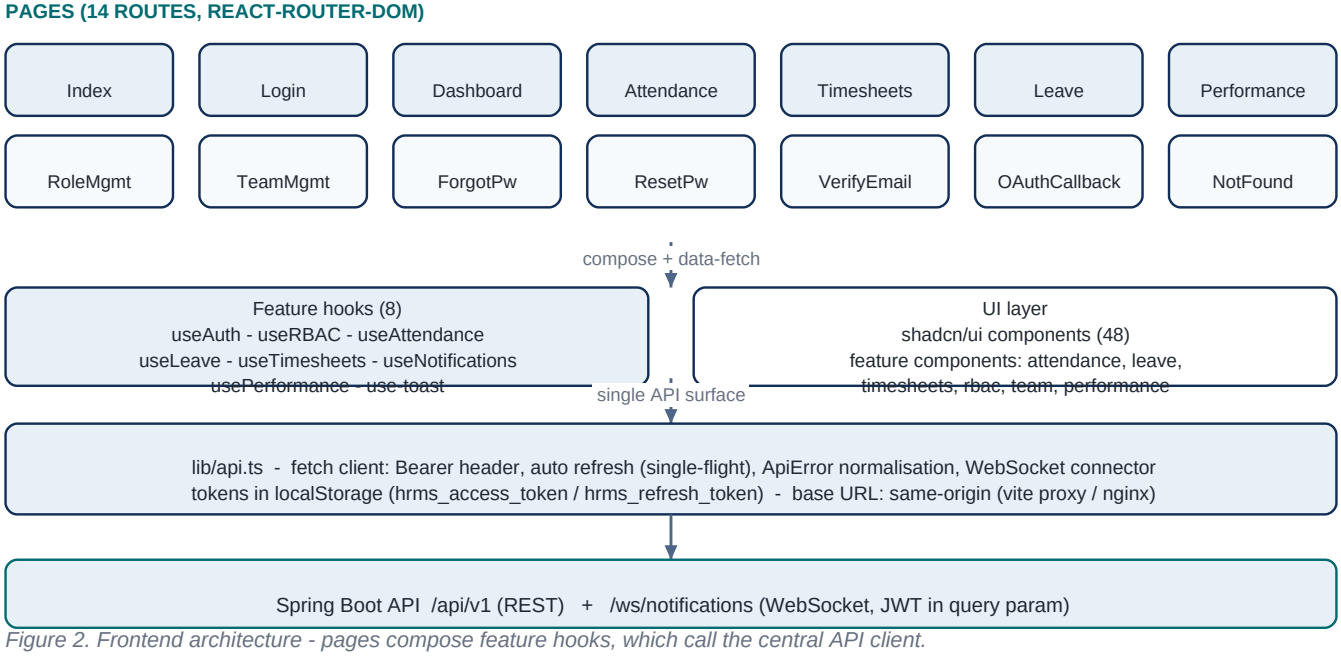


Figure 2. Frontend architecture - pages compose feature hooks, which call the central API client.

9.1 Routing, State and Communication

- Routing (App.tsx):
  14 routes under BrowserRouter with a catch-all NotFound. Pages are thin wrappers composing feature components (e.g. Login renders LoginPage). No route-level code splitting is configured - all pages ship in one bundle.
- Server state:
  TanStack Query QueryClientProvider wraps the app; feature hooks (useAttendance, useLeave, useTimesheets, useNotifications, usePerformance) encapsulate queries/mutations against /api/v1.
- Session state:
  useAuth loads /auth/me on mount, stores user/profile/role, exposes signIn/signUp/signOut and OAuth redirect helpers; a 401 clears local tokens.
- Authorization UI:
  useRBAC reads the permission list returned by /auth/me to gate navigation and actions client-side; the server remains the enforcement point.
- Real-time:
  connectNotifications opens /ws/notifications?token=JWT, retries every 5 s on close, and feeds the notification bell.
- Forms & validation:
  react-hook-form + zod resolvers are available; timesheet/leave screens validate inputs before submission; server-side validation stays authoritative.
- Error handling:
  ApiError (status + code + message) is thrown for non-2xx responses; toasts surface messages; auto-refresh retry happens at most once per request.
- Theming:
  ThemeProvider defaults to dark; a theme-toggle component exists; design tokens flow through Tailwind configuration.

10 Backend Architecture

10.1 Structure and Layering

The backend is a single Maven module (com.hrms) organised package-by-feature. Each business module is self-contained: controller (HTTP + DTO records), service (business rules + transactions), entities and Spring Data repositories. Cross-cutting concerns live in config/, security/, common/ and events/.

Layer	Implementation	Notes
Web / controllers	@RestController classes with @RequestMapping("/api/v1/...")	DTOs declared as Java records inside controllers; Swagger @Operation annotations
Security filters	RateLimitFilter -> JwtAuthenticationFilter (both before UsernamePasswordAuthenticationFilter)	Stateless sessions; CSRF disabled (bearer model); CORS from AppProperties
Services	@Service + @Transactional business logic; RBAC via rbac.requirePermission	Business failures thrown as ApiException (400/403/404/409/rules)
Persistence	Spring Data JPA repositories; Hibernate ddl-auto: validate; Flyway owns schema	Open-in-view disabled; UTC timezone enforced
Events	OutboxService.append (MANDATORY propagation) + scheduled dispatcher + @KafkaListener consumer	Topic hrms.events; DLT hrms.events.dlt; JSON serde
Async / scheduling	Outbox dispatch poller, purge job, AttendanceReminderJob (@Scheduled)	Email and reminder work never blocks API threads
Real-time	WebSocketConfig registering NotificationWebSocketHandler at /ws/notifications	JWT passed as query parameter; per-user push
Configuration	AppProperties @ConfigurationProperties; RedisConfig; KafkaConfig + KafkaErrorConfig; OpenApiConfig; CorrelationIdFilter	All secrets from environment variables (Appendix)
Error handling	GlobalExceptionHandler -> ErrorResponse {code, message}; stack traces disabled	Machine-readable codes (EMAIL_NOT_VERIFIED, RATE_LIMIT_EXCEEDED, ...)
Observability	Actuator (health/info/metrics/prometheus); Micrometer counters; Logback MDC (correlationId, userId)	Rolling daily log files, 14-day retention, logs/hrms.log

10.2 Business Logic Highlights

- Attendance:
  checkIn rejects a second record for the same day (a DB unique constraint backs the guard); status becomes 'late' when LocalTime.now() exceeds the active policy start plus grace minutes; checkout computes total hours.
- Leave:
  createRequest validates active type, date ordering, overlapping pending/approved requests (repository existsOverlapping) and remaining balance before persisting; rejection requires a reason.
- Timesheets:
  entries are only editable on draft/rejected sheets owned by the caller; every entry change recalculates the sheet total; decisions emit events for notification.
- RBAC:
  effectiveRole merges the legacy user_roles enum with dynamic assignments; permissions load from role_permissions, are cached in Redis for 5 minutes, and are invalidated on any role/assignment change.
- Notifications:
  every notification side-effect flows through the outbox -> Kafka -> consumer path with Redis deduplication on event IDs and a dead-letter topic for poison messages.

11 API Documentation

Base URL: /api/v1 (backend port 8081; proxied same-origin in production). 'Auth' column: Public = reachable without a token; JWT = valid access token required; Perm = additionally gated by an RBAC module:action permission. Interactive docs are served at /swagger-ui.html (springdoc).

Method	Endpoint	Purpose	Auth	Errors
POST	/api/v1/auth/signup	Create account, send verification email	Public	409 duplicate
POST	/api/v1/auth/login	Email/password sign-in -> token pair	Public + rate limit	401, 403 (locked /unverified)
POST	/api/v1/auth/refresh	Rotate refresh token -> new pair	Public	401 invalid/expired
POST	/api/v1/auth/logout	Revoke refresh token	Public	400
POST	/api/v1/auth/forgot-password	Email a reset link (no user enumeration)	Public	200 always
POST	/api/v1/auth/reset-password	Set new password with one-time token	Public	400 invalid/expired
POST	/api/v1/auth/change-password	Change password while signed in	JWT	400 wrong current
POST	/api/v1/auth/verify-email	Verify email with token param	Public	400 invalid/expired
POST	/api/v1/auth/resend-verification	Re-send verification email	Public	200 (idempotent)
GET	/api/v1/auth/me	Current user, profile, role, permissions	JWT	404
GET	/api/v1/attendance/today	My attendance for today	JWT	404 profile
GET	/api/v1/attendance/me?start&end	My attendance history (range)	JWT	400 parse
POST	/api/v1/attendance/check-in	Clock in (notes optional)	JWT	409 already checked in
POST	/api/v1/attendance/check-out	Clock out, compute total hours	JWT	400/409 state
POST	/api/v1/attendance/break/start	Start break window	JWT	400/409 state
POST	/api/v1/attendance/break/end	End break window	JWT	400/409 state
POST	/api/v1/attendance/corrections	Request attendance correction	JWT	400/404
GET	/api/v1/attendance/corrections	Pending corrections (approver)	Perm attendance :approve	403
POST	/api/v1/attendance/corrections/{id}/approve	Approve/amend correction	Perm attendance :approve	403/404
POST	/api/v1/attendance/corrections/{id}/reject	Reject correction	Perm attendance :approve	403/404
GET	/api/v1/attendance/report/{profileId}?year&month	Monthly attendance report	JWT	404
GET/POST	/api/v1/attendance/holidays	List (by year) / create holiday	JWT / Perm attendance:edit	403
PUT/DELETE	/api/v1/attendance/holidays/{id}	Update / delete holiday	Perm attendance:edit	403/404
GET	/api/v1/attendance/policies	Active attendance policies	JWT	-
POST/PUT/DELETE	/api/v1/attendance/policies(/{id})	Policy administration	Perm attendance:edit	403/404
GET	/api/v1/attendance/team?start&end	Team attendance (non-employees)	JWT (role check)	403

Table 1. Consolidated REST surface (56 method/route groups across 8 controllers plus WebSocket). OAuth2 login endpoints (/oauth2/authorization/{google|github} and /login/oauth2/code/) are provided by Spring Security.*

Method	Endpoint	Purpose	Auth	Errors
GET	/api/v1/employees?search&department&page&size	Paged employee search	JWT	-
GET	/api/v1/employees/active | /me | /{id}	Active list / own profile / by id	JWT	404
POST	/api/v1/employees	Create employee	Perm role_management:assign	409 duplicate
PATCH	/api/v1/employees/{id}	Update / onboard / offboard	Perm role_management:edit	404
GET	/api/v1/leaves/types | /balances?year	Leave catalogue and my balances	JWT	-
GET	/api/v1/leaves/requests	Role-aware leave list	JWT	-
POST	/api/v1/leaves/requests	Submit leave request	Perm leave_management:create	400/409 balance/overlap
POST	/api/v1/leaves/requests/{id}/decision	Approve / reject (reason if rejected)	Perm leave_management:approve	400/403
DELETE	/api/v1/leaves/requests/{id}	Cancel pending request	JWT (owner)	404/409
POST/PUT/DELETE	/api/v1/leaves/types(/{id})	Leave type administration	Perm role_management:create/edit/delete	403/409
POST	/api/v1/leaves/balances	Ensure/update a leave balance	JWT (HR/admin)	404
GET	/api/v1/timesheets	Role-aware timesheet list	JWT	-
POST	/api/v1/timesheets/current	Get-or-create week sheet	JWT	-
POST/DELETE	/api/v1/timesheets/{id}/entries(/{entryId})	Add/update/remove entry	JWT (owner, draft/rejected)	403/404/409
POST	/api/v1/timesheets/{id}/submit	Submit for approval	JWT (owner)	409 state
POST	/api/v1/timesheets/{id}/decision	Approve / reject with reason	Perm timesheets:approve	400/403/409
POST	/api/v1/timesheets/bulk-approve	Bulk approve submitted sheets	Perm timesheets:approve	403
GET	/api/v1/reports/performance?department	Weekly performance scorecards	Perm attendance:view	403
GET	/api/v1/reports/performance/{profileId}/detail	Trends + workload drill-down	Perm attendance:view	403/404
GET	/api/v1/roles | /permissions	List roles / permissions	JWT	-
POST/PUT/DELETE	/api/v1/roles(/{id})	Role administration	Perm role_management:create/edit/delete	403/409/404
GET/POST/DELETE	/api/v1/role-permissions(/{roleId}/{permissionId})	View / grant / revoke role permissions	Perm role_management:edit	403/404
GET/POST/DELETE	/api/v1/user-roles/{userId}(/{roleId})	Assign / list / remove user roles	Perm role_management:assign	403/404/409
GET	/api/v1/users-with-roles	All users with assignments	Perm role_management:view	403
GET	/api/v1/me/permissions	My effective permissions	JWT	-
GET	/api/v1/notifications | /unread-count	List (top 50) + unread count	JWT	-
POST	/api/v1/notifications/{id}/read | /read-all	Mark read / mark all read	JWT	-
GET	/api/v1/admin/audit-logs?page&size	Paged audit trail	Perm role_management:view	403
GET	/actuator/health | /actuator/prometheus	Health + metrics scrape	Public (intentional)	-
WS	/ws/notifications?token=JWT	Live notification push	JWT (query param)	-

12 Database Architecture

PostgreSQL 16 with a Flyway-managed schema (V1__init.sql creates 22 tables, V2__seed.sql seeds 20 permissions, 4 system roles with permission grants, 4 leave types and a default attendance policy). Hibernate runs with ddl-auto: validate, so any drift between entities and migrations fails startup. The migration header notes the schema was converted from the original Supabase migrations, which remain under supabase/migrations.

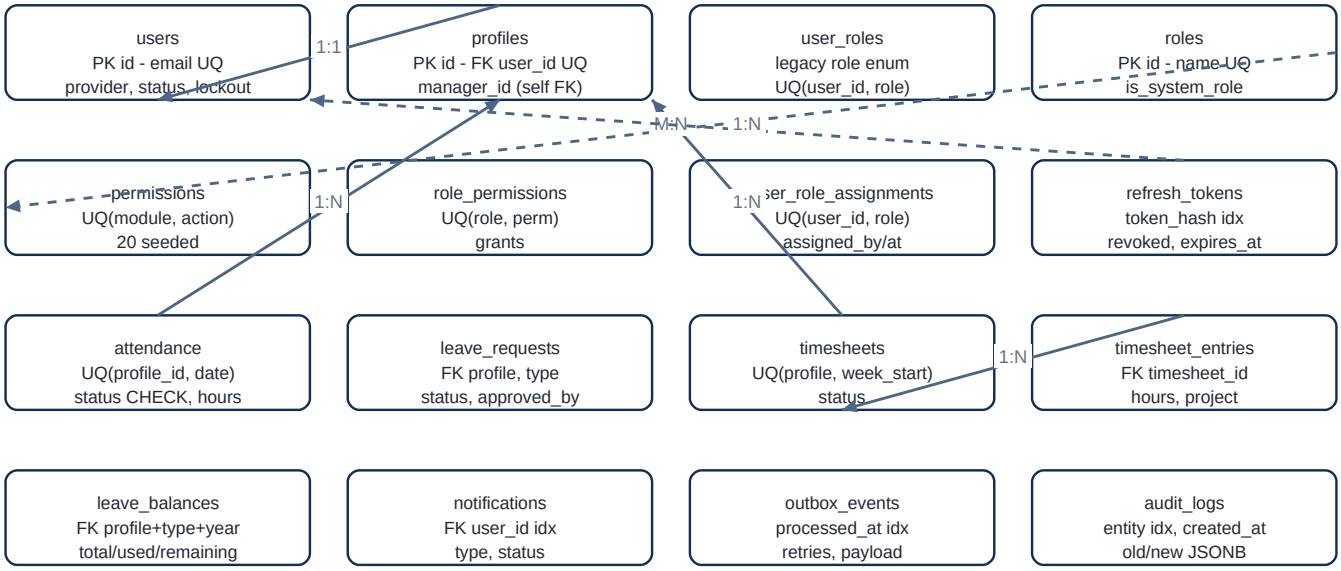


Figure 3. Simplified entity-relationship view (16 of 22 tables; attendance_policies, holidays, leave_types, one_time_tokens and the three audit tables are detailed below).

12.1 Table Inventory (V1__init.sql)

Table	Purpose	Key Columns / Constraints
users	Identity (replaces Supabase auth.users)	id PK, email UQ, password (bcrypt), provider CHECK(LOCAL/GOOGLE/GITHUB), status CHECK, failed_login_attempts, locked_until
profiles	Employee record 1:1 with user	user_id UQ FK->users CASCADE, employee_id UQ, manager_id self-FK, department, hire_date, is_active
user_roles	Legacy quick roles (mirrors Supabase)	role CHECK(SUPER_ADMIN/HR/MANAGER/EMPLOYEE), UQ(user_id, role)
roles / permissions	Dynamic RBAC catalogue	roles.name UQ; permissions UQ(module, action); 4 roles + 20 permissions seeded
role_permissions / user_role_assignments	Grants and assignments	UQ(role_id, permission_id); UQ(user_id, role_id), assigned_by, assigned_at
audit_logs (+ rbac, attendance variants)	Append-only trails	JSONB old/new values; indexes on created_at and entity
attendance	Daily attendance	UQ(profile_id, date), status CHECK(present/absent/late/half_day), total_hours DECIMAL(4,2), corrected_by FK
attendance_policies / holidays	Policy configuration	09:00-17:00 defaults, grace 15 min, overtime threshold; holidays.date idx, is_recurring, year
leave_types / leave_balances / leave_requests	Leave domain	max_days_per_year, carry_forward; balances UQ per profile+type+year; requests status CHECK, approved_by / rejection_reason
timesheets / timesheet_entries	Weekly timesheets	UQ(profile_id, week_start_date); entries FK timesheet_id, hours, project_name, is_billable
refresh_tokens / one_time_tokens	Token storage	SHA-256 token_hash idx, expires_at idx, revoked/revoked_at; purpose, used_at
notifications	In-app + email records	user_id idx, (user_id, status) idx, type, status, read_at

Table	Purpose	Key Columns / Constraints
outbox_events	Transactional outbox	processed_at idx, retries, payload JSONB

12.2 Integrity, Indexing and Security Policies

- Constraints:
  CHECK constraints encode enums (attendance.status, users.provider/status, user_roles.role); UNIQUE keys prevent duplicate emails, duplicate daily attendance and duplicate role grants/assignments; optimistic-lock version columns on hot entities.
- Indexes:
  idx_attendance_profile_date, idx_attendance_date, idx_leave_balances_profile, idx_leave_requests_(profile,status), idx_timesheet_entries_(timesheet,date), idx_refresh_tokens_(user_id,expires_at), idx_outbox_unprocessed, audit created_at/entity indexes.
- Security policy shift:
  the original Supabase row-level security was replaced by application-layer RBAC (RbacService.requirePermission at every privileged endpoint). No RLS policies exist in the Flyway schema.
- Migration discipline:
  ddl-auto: validate guarantees schema/entity alignment at startup; migrations are versioned (V1 schema, V2 seed).

13 Authentication and Authorization

AUTHENTICATION FLOW

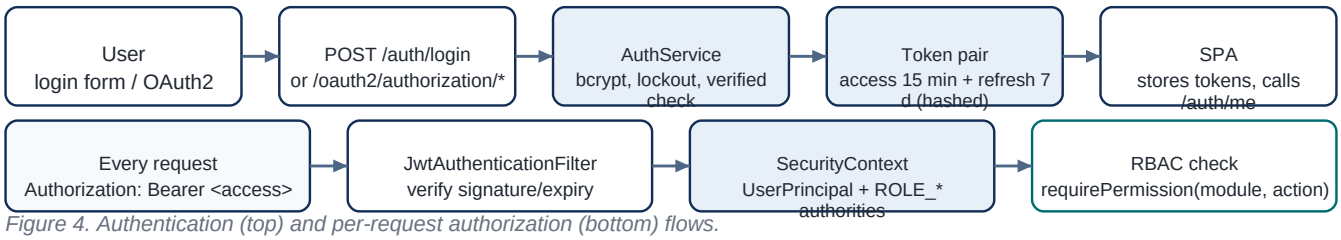


Figure 4. Authentication (top) and per-request authorization (bottom) flows.

13.1 Mechanisms in Detail

Aspect	Implementation	Evidence
Registration	Email+password with case-insensitive uniqueness; creates user + profile; assigns default Employee role in both role systems; emits verification email event.	AuthService.signUp; RbacService.assignDefaultRole
Password handling	BCrypt with strength 12; Supabase bcrypt hashes are portable into users.password per setup guide.	SecurityConfig.passwordEncoder
Email verification	32-byte SecureRandom token, SHA-256 hashed at rest, purpose + expiry; sign-in rejected with EMAIL_NOT_VERIFIED until verified.	one_time_tokens; AuthService.signIn
Session model	Stateless JWT; no server sessions (SessionCreationPolicy.STATELESS).	SecurityConfig
Refresh rotation	Refresh token stored as SHA-256 hash; refresh exchanges old for new (old revoked); logout and password reset revoke all sessions.	AuthService.refresh / revokeAllSessions
Account protection	5 failed attempts -> 15-minute lock (users.locked_until); Redis rate limit 20/min per IP on auth endpoints.	AuthService constants; RateLimitFilter
OAuth2	Google (openid, profile, email) and GitHub (read:user, user:email) with server-side callbacks; custom user service provisions locals; success handler issues JWT pair.	SecurityConfig; application.yml oauth2 block
Authorization	Effective role from legacy enum or dynamic assignments (super_admin > hr > manager > employee); module:action permissions checked per endpoint; 403 JSON via accessDeniedHandler.	RbacService; SecurityConfig handlers
Frontend protection	Routes rendered only with a valid session (useAuth); role-based navigation via useRBAC; 401 clears tokens and returns to login.	useAuth.ts, useRBAC.ts
Logout	POST /auth/logout revokes the refresh token; client clears localStorage.	AuthService.logout; useAuth.signOut

14 Complete Application Workflows

14.1 Leave Request Workflow

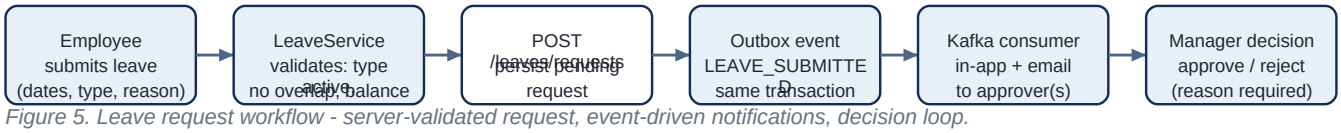


Figure 5. Leave request workflow - server-validated request, event-driven notifications, decision loop.

14.2 Timesheet Workflow (state machine)

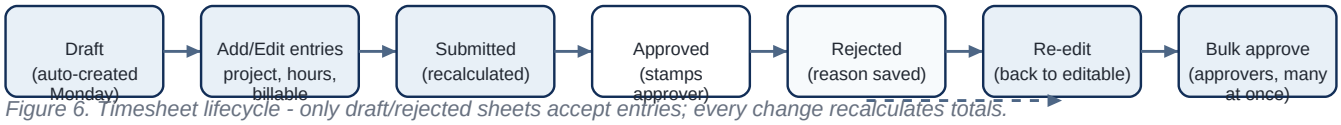


Figure 6. Timesheet lifecycle - only draft/rejected sheets accept entries; every change recalculates totals.

14.3 Attendance Daily Cycle

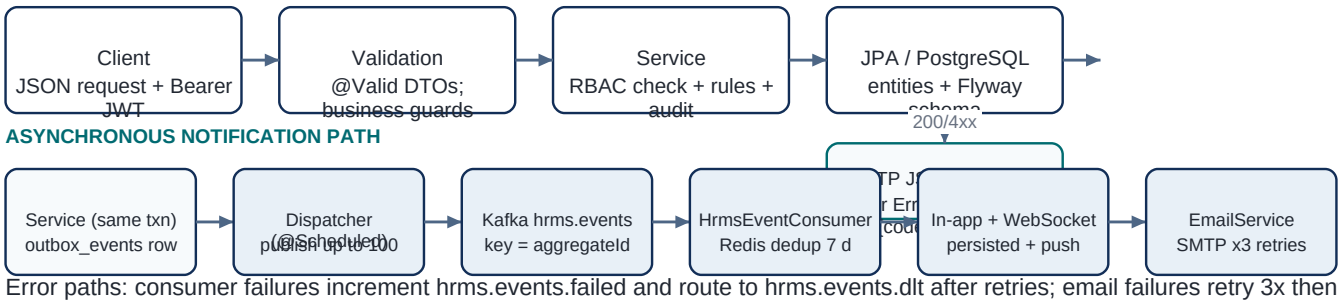
- Clock in ->
  guard against duplicate day record; status = late if now > policy start + grace; audit row written.
- Break start/end ->
  windows recorded on the same daily record.
- Clock out ->
  total hours computed (checkout minus check-in minus break); record becomes final for the day.
- Correction ->
  employee requests with reason; approver amends times/status; audit snapshot + ATTENDANCE_CORRECTED event notify the employee.

14.4 Signup-to-Working-Account Flow

- Sign up -> verification email (async via outbox/Kafka) -> click token link (VerifyEmail page) -> account verified.
- Sign in -> tokens stored -> /auth/me returns profile, effective role, permissions -> role-aware dashboard.
- First administrator is promoted manually via SQL (documented in README-SETUP.md): UPDATE user_roles SET role = 'SUPER_ADMIN' ... plus an RBAC assignment row.

15 Complete Data Flow

SYNCHRONOUS WRITE PATH



Error paths: consumer failures increment hrms.events.failed and route to hrms.events.dlt after retries; email failures retry 3x then log permanently; outbox dispatch failures increment retries and are retried on the next poll; Redis unavailability degrades (permission lookups fall back to the database; rate limiting skips when Redis errors).

Figure 7. Data flow - synchronous request path (top) and asynchronous event pipeline (bottom).

16 Project Folder Structure

Root: c:\Users\Mukesh\Desktop\HRMS\slate-timesync

```
slate-timesync/
  README.md                Lovable boilerplate (outdated - see README-SETUP.md)
  README-SETUP.md          Accurate setup/verification guide for this stack
  .env.example             All environment variables (names + safe examples)
  .gitignore               node_modules, dist, .env, target, logs...
  backend/                 Spring Boot 3.3 modular monolith (Java 21, Maven)
    Dockerfile             multi-stage: Maven build -> JRE 21 alpine runtime
    src/main/java/com/hrms/
      HrmsApplication.java  entry point (@SpringBootApplication)
      auth/                AuthController, AuthService, OAuth2 handlers, DTOs, repos
      security/            JwtTokenProvider, JwtAuthenticationFilter, RateLimitFilter,
                           SecurityUtils, UserPrincipal
      config/              SecurityConfig, AppProperties, RedisConfig, KafkaConfig,
                           KafkaErrorConfig, OpenApiConfig, CorrelationIdFilter
      employee/            EmployeeController/Service, Profile, ProfileRepository
      attendance/          controller, service, entities (attendance, policies,
                           holidays, audit), repositories, AttendanceReminderJob
      leave/               LeaveController/Service, LeaveType/Balance/Request, repos
      timesheet/           TimesheetController/Service, Timesheet/Entry, repos
      reports/             ReportController (performance metrics)
      role/                RbacController/Service, RoleEntity, PermissionEntity, grants
      notification/        NotificationController/Service, WebSocket handler + config
      email/               EmailService (SMTP + retry), EmailTemplates
      events/              OutboxService/Event/Repository, HrmsEvent,
                           HrmsEventConsumer, OutboxDispatcher
      audit/               AuditController, AuditLog, AuditRecorder, AuditRepository
      common/exception/    ApiException, ErrorResponse, GlobalExceptionHandler
    src/main/resources/
      application.yml      datasource, redis, kafka, mail, oauth2, hrms.* props
      logback-spring.xml   console + rolling file (14 days), MDC pattern
      db/migration/        V1__init.sql (22 tables), V2__seed.sql (seed data)
    src/test/java/com/hrms/ AuthServiceTest, JwtTokenProviderTest, FlywayMigrationTest
  frontend/                React 18 + TypeScript + Vite SPA
    Dockerfile, nginx.conf  nginx:1.27 hosting + /api,/ws reverse proxy
    vite.config.ts         proxies /api,/oauth2,/login,/ws to :8081 in dev
    src/
      App.tsx              providers + 14 routes
      lib/api.ts           fetch client: tokens, refresh rotation, ApiError, WebSocket
      hooks/               useAuth, useRBAC, useAttendance, useLeave, useTimesheets,
                           useNotifications, usePerformance, use-toast
      pages/               Dashboard, Attendance, Timesheets, Leave, RoleManagement,
                           TeamManagement, Performance, Login, Index, password flows...
      components/          feature components + ui/ (48 shadcn components)
  docker/
    docker-compose.yml     postgres, redis, kafka, kafka-ui, backend, frontend,
                           prometheus, grafana
    prometheus/prometheus.yml, grafana/ (provisioning + hrms.json dashboard)
  supabase/                legacy (pre-migration) schema history + 5 Deno edge functions
    migrations/            original 8 SQL migrations (schema history)
    functions/             check-* / send-* email reminder functions
```

17 Module and Implementation Details

Module	Responsibility	Main Logic	Depends On
auth	Account lifecycle and token issuance	signUp/signIn/refresh/logout/verify/reset flows; lockout; OAuth provisioning; me() aggregation	security, role (RBAC), events (outbox), employee (profile)
security	Request authentication + throttling	JWT verify per request; Redis rate counters; client-IP resolution (X-Forwarded-For first hop)	config (AppProperties), Redis
attendance	Daily attendance + corrections + policy admin	grace-period lateness, total-hours computation, snapshot auditing, correction approval	events, employee (profile)
leave	Leave governance	4 validation guards, decisions, balance admin, role-aware visibility	role, events
timesheet	Weekly timesheet cycles	Monday anchoring, entry CRUD with state guard, recalculation, bulk approve	role, events
employee	Directory management	paged search, EMP-id generation, onboarding/offboarding, audit	audit, role
role (RBAC)	Dynamic permissions	effective-role precedence, Redis permission cache (5 min), admin operations with cache invalidation	employee, Redis
events	Reliable async notifications	transactional outbox + Kafka publish/consume + DLT + metrics	email, notification, employee, leave, attendance
notification	In-app channel	persist + WebSocket push + read state	WebSocket handler
email	SMTP delivery	3 attempts linear backoff; disabled-mode logging; Micrometer counters	mail config, AppProperties
audit	Privileged-change trail	append-only recorder + admin paged query	role (permission gate)
reports	Performance analytics	weekly scorecard + drill-down aggregations	employee, attendance, timesheet, role
common.exception	Uniform error contract	ApiException factories (badRequest, notFound, duplicate, forbidden, businessRule) -> handler	Spring web
config	Cross-cutting wiring	CORS allow-list, Redis, Kafka (JSON + DLT), OpenAPI, correlation filter	AppProperties

18 Validation and Error Handling

18.1 Backend

- Bean Validation:
  @Valid on request bodies (AuthController, LeaveController, TimesheetController) with jakarta constraints (@NotNull, @NotBlank) on DTO records; violations handled by GlobalExceptionHandler.
- Business validation:
  explicit guards in services - duplicate email/employee ID, already-checked-in/out, overlapping leave, insufficient balance, invalid state transitions (timesheet status), missing rejection reasons.
- Database validation:
  CHECK + UNIQUE + FK constraints as the final backstop; Hibernate validate prevents schema drift at boot.
- Error contract:
  ErrorResponse {code, message} JSON; ApiException factories map intent to status (400, 403, 404, 409, business-rule failures); AccessDenied -> 403; JSON 401 entry point; include-stacktrace: never.
- External service failures:
  email retries 3x with 1s/2s/3s backoff then permanent-failure log + metric; Kafka consumer uses ErrorHandlingDeserializer and a DLT; outbox poller retries with a retry counter; Redis failures degrade gracefully (cache miss / rate-limit skip) with warnings.
- Logging and correlation:
  CorrelationIdFilter propagates X-Correlation-Id; JwtAuthenticationFilter adds userId to MDC; every line carries timestamp, level, app, correlationId, user; rolling file logs with 14-day history.

18.2 Frontend

- Single fetch wrapper converts all non-2xx responses into ApiError(status, code, message); 204 passthrough.
- On 401 with a stored refresh token, exactly one refresh attempt (single-flight promise) then a single retry.
- UI feedback via toast notifications on mutation success/failure; destructive variant for errors.
- WebSocket client auto-reconnects every 5 s and silently ignores malformed frames.

19 Security Analysis

19.1 Implemented Security

Control	Implementation	Assessment
Password storage	BCrypt strength 12	Strong
Token storage	Refresh + one-time tokens stored as SHA-256 hashes; refresh rotation; revocation on logout/reset	Strong
Brute-force defence	5-attempt lockout (15 min) + Redis rate limiting (20/min auth, 300/min API)	Strong
Authorization	RBAC module:action checks on every privileged endpoint; 403 JSON handler	Strong
Secrets management	All credentials via environment variables; .env gitignored; .env.example has placeholders only	Good
CORS	Explicit allow-list from CORS_ALLOWED_ORIGINS; credentials enabled	Good
Error exposure	include-stacktrace: never; JSON 401/403 handlers; no user enumeration on forgot-password	Good
Injection	JPA parameter binding throughout; no string-concatenated SQL identified	Good
XSS	React escapes output by default; no dangerouslySetInnerHTML usage identified in reviewed sources	Baseline good
Auditability	Append-only audit tables with actor + old/new snapshots; admin-only access	Strong
CSRF	Disabled; justified by stateless bearer-token model (no cookies)	Acceptable
Mass assignment	DTO records with explicit fields instead of direct entity binding	Good

19.2 Recommended Security Improvements

Finding	Risk	Recommendation
JWT secret has a committed fallback in application.yml (change-me-in-production-...). If JWT_SECRET is unset in production, tokens are signed with a publicly-known key.	High	Fail startup when JWT_SECRET is missing or equals the default; add a startup assertion.
RateLimitFilter and AuthController trust the first X-Forwarded-For value, which a client can spoof; limits can be bypassed.	High	Trust proxy headers only from known proxies (ForwardedHeaderFilter / trusted-proxy config).
JWT passed as a WebSocket query parameter may appear in access logs.	Medium	Use Sec-WebSocket-Protocol or a short-lived single-use ticket.
Tokens stored in localStorage are readable by any injected script.	Medium	Consider httpOnly Secure cookies for the refresh token; keep short-lived access in memory.
No security response headers (HSTS, CSP, X-Content-Type-Options, frame-ancestors) in nginx or Spring.	Medium	Add a headers block in nginx.conf and/or Spring Security headers DSL.
Swagger UI, /api-docs and /actuator/prometheus are public.	Medium	Restrict by network or authentication in production.
No dependency vulnerability scanning (OWASP dependency-check / npm audit in CI) - not identified in the current implementation.	Medium	Add dependency scanning to CI.
No JWT key-rotation mechanism; changing the secret invalidates all sessions (undocumented).	Low	Document rotation; consider kid-based rotation.
Rate limiter is per-IP only; authenticated abuse across many IPs is unthrottled.	Low	Add a per-user limit for authenticated traffic.

Secret exposure check: the repository contains only placeholder secrets (.env.example, compose defaults). No real API keys, passwords or tokens were found in the inspected files, and none are reproduced in this report.

20 Testing

20.1 Test Inventory (actual files in backend/src/test)

Test Class	Type	Tests	What It Covers
AuthServiceTest	Unit (JUnit 5 + Mockito)	7	Duplicate signup rejection; signup creates user/profile + verification event; wrong password increments failure counter; lockout after 5 failures; unverified email rejected (EMAIL_NOT_VERIFIED); forgot-password does not enumerate users; refresh rotation revokes the old token
JwtTokenProviderTest	Unit	2	Token create/parse round-trip with roles; tampered and malformed tokens rejected
FlywayMigrationTest	Integration (@SpringBootTest + Testcontainers PostgreSQL 16)	1	Flyway migrations run against a real database; seed applied; effectiveRole falls back to employee; fresh database empty. Auto-skips without Docker (disabledWithoutDocker = true)

20.2 Test Execution in the Authoring Environment

The authoring environment has Node.js 24 / npm 11 installed but not Java 21, Maven, or Docker. Consequently mvn test could not be executed, and the Testcontainers-based integration test would additionally require Docker. The 10 backend tests are documented from source inspection; their pass/fail status is NOT claimed. This is an environment limitation, not a project defect - the test tooling is correctly wired (spring-boot-starter-test, spring-security-test, testcontainers junit-jupiter/postgresql, spring-boot-testcontainers, H2 for fast contexts).

20.3 Frontend Testing

No test runner, test files or test scripts are configured in frontend/package.json (scripts: dev, build, build:dev, lint, preview only). E2E or component tests are not identified in the current implementation. Frontend verification therefore relied on production build and lint, reported in Section 21.

21 Verification and Quality Assurance

The following checks were actually executed during report preparation on the real codebase. Anything not executable in the authoring environment is explicitly marked.

Verification	Command / Method	Result	Status
Frontend production build	npm run build (vite v5.4.19)	3822 modules transformed; built in 8.63 s; warning: main chunk 1,921 kB (>500 kB)	PASS
Frontend static analysis	npm run lint (eslint 9)	128 problems (100 errors, 28 warnings), e.g. require() import in tailwind.config.ts	FAIL
Backend unit/integration tests	mvn test	Not executable - Java 21/Maven absent in authoring environment	Not executed
Backend compile	mvn package	Not executable - Java 21/Maven absent	Not executed
Full stack startup	docker compose -f docker/docker-compose.yml up	Not executable - Docker absent	Not executed
Schema review	Manual inspection of V1__init.sql + V2__seed.sql	22 tables, FKs, CHECK/UNIQUE constraints, seed roles/permissions verified	PASS
Configuration consistency	Cross-check .env.example vs application.yml vs compose	All variable names align (DATABASE_*, JWT_*, MAIL_*, OAuth, CORS)	PASS
API surface review	Mapping annotation scan of 8 controllers	56 method/route groups documented (Section 11)	PASS
Secrets scan	Inspection of committed files (.env.example, compose, configs)	Placeholders only; no real credentials found	PASS
Dependency inventory	package.json + pom.xml review	Documented in Section 6 / Appendix	PASS

22 Performance Analysis

No load testing or runtime benchmarking was possible in the authoring environment (no runtime), so measured numbers are limited to what was actually observed; the rest is inspection-based analysis.

Aspect	Observation / Evidence	Basis
Frontend build performance	Production build: 3822 modules, 8.63 s total, 70 kB CSS, 1,921 kB main JS (570 kB gzip)	Measured (npm run build)
Bundle size risk	Main JS chunk exceeds 500 kB; Vite recommends manualChunks / dynamic import(). No route-level code splitting configured.	Measured + Vite warning
Database access	Hikari pool max 10; hot queries covered by composite indexes (attendance profile+date, timesheet entries by sheet/date, refresh tokens by expiry).	Code inspection
Cache behaviour	Permissions cached 5 min per user in Redis; generic Spring cache TTL 300 s; RBAC lookups avoid a 3-table join per request.	Code inspection
Potential hot spots	Team/hr listing paths use findAll().limit(200) style reads; HrmsEventConsumer emails all top-level profiles per TIMESHEET_SUBMITTED event; performance report aggregates per-profile in a loop.	Code inspection
Rate-limit overhead	One Redis INCR (+EXPIRE on first hit) per API request; cheap but adds a Redis round-trip per call.	Code inspection
Async offloading	Email and notification fan-out are fully off the request thread (outbox + Kafka), keeping write latency dominated by the DB transaction only.	Code inspection

23 Scalability

Dimension	Behaviour as Load Grows	Bottlenecks / Remedies
Concurrent users	Stateless API (JWT) allows adding backend replicas behind any load balancer; no sticky sessions needed for REST.	WebSocket connections are per-instance; sticky routing or a Redis/Kafka-backed WS broker needed at scale.
API traffic	Rate limiting shared via Redis works across replicas.	Monolith scaling is coarse; modules are cleanly bounded for future extraction.
Data volume	Indexes cover primary query paths; pagination on employees/audit.	findAll().limit(200) paths (leave/timesheet listing for HR) will degrade; replace with paged queries. Outbox purge (7 days) exists; audit tables lack retention.
Event throughput	Kafka partitions by aggregateId; consumer group scales horizontally.	Single consumer group with default concurrency; raise concurrency per replica as volume grows.
Notification fan-out	Per-event emails batched per consumer record; idempotent via Redis.	Timesheet-submitted fan-out to all top-level profiles is O(approvers); target direct managers instead.
Database	Single PostgreSQL node.	Read replicas / managed Postgres when write load grows; pool (10) should scale with replicas.

24 Deployment and DevOps

DOCKER COMPOSE - 8 SERVICES (DOCKER/DOCKER-COMPOSE.YML)

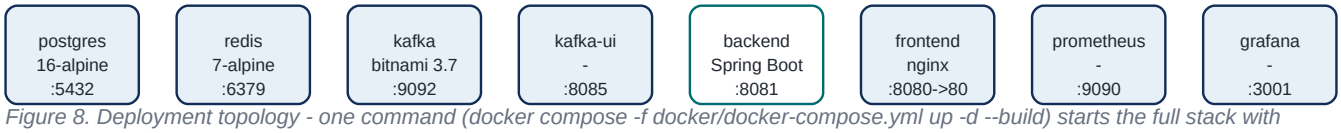


Figure 8. Deployment topology - one command (docker compose -f docker/docker-compose.yml up -d --build) starts the full stack with healthchecks and depends_on ordering.

- Build images:
  backend multi-stage (maven:3.9-temurin-21 build -> eclipse-temurin:21-jre-alpine, MaxRAMPercentage=75); frontend multi-stage (node:20-alpine build -> nginx:1.27-alpine).
- Runtime wiring:
  nginx proxies /api and /ws to backend:8081; backend depends_on postgres/redis (healthy) and kafka (started); kafka-ui at :8085 for topic inspection.
- Observability deployment:
  Prometheus scrapes /actuator/prometheus (docker/prometheus/prometheus.yml); Grafana auto-provisions the datasource and a pre-built HRMS dashboard (docker/grafana/dashboards/hrms.json).
- Environments:
  single compose profile for local/self-hosted deployment; environment variables injected from .env (Appendix). Kubernetes or CI/CD manifests are not identified in the current implementation.
- Legacy channel:
  supabase/functions document the former Supabase deployment path (Deno edge functions), retained for history.

25 Installation and Setup

Verified against README-SETUP.md, .env.example and the Dockerfiles. Prerequisites: Java 21, Maven 3.9+ (or Docker), Node 20/npm, Docker Desktop (for Postgres, Redis, Kafka, Prometheus, Grafana).

Option A - full stack in Docker

```
cp .env.example .env      # fill DATABASE_*, JWT_SECRET, MAIL_*, OAuth ids
docker compose -f docker/docker-compose.yml up -d --build
# Frontend   http://localhost:8080     Backend API  http://localhost:8081/api/v1
# Swagger    http://localhost:8081/swagger-ui.html
# Prometheus http://localhost:9090     Grafana      http://localhost:3001 (admin)
# Kafka UI   http://localhost:8085
```

Option B - local development

```
docker compose -f docker/docker-compose.yml up -d postgres redis kafka
cd backend  && mvn spring-boot:run          # needs JDK 21
cd frontend && npm install && npm run dev   # vite proxies /api -> :8081
```

First administrator (from README-SETUP.md)

```
UPDATE user_roles SET role = 'SUPER_ADMIN'
WHERE user_id = (SELECT user_id FROM profiles WHERE email = 'you@company.com');
INSERT INTO user_role_assignments (user_id, role_id)
SELECT u.id, r.id FROM users u, roles r
WHERE u.email = 'you@company.com' AND r.name = 'Super Admin';
```

OAuth, email and production build

- Google Cloud Console OAuth web client; redirect URI http://localhost:8081/login/oauth2/code/google.
- GitHub OAuth App callback http://localhost:8081/login/oauth2/code/github.
- SendGrid SMTP relay: host smtp.sendgrid.net, user 'apikey', password = SG key; MAIL_FROM must be a verified sender; set MAIL_ENABLED=false to log emails instead of sending during development.
- Production build: mvn -DskipTests package (backend jar) and npm run build (frontend dist served by nginx).

26 Problems Found and Improvements

No historical fix log exists in the repository, so every item below is a potential issue identified during this analysis (Problem -> Cause -> Recommended Solution). None is claimed as already fixed.

Problem	Cause	Recommended Solution
Outdated root README (Lovable boilerplate) contradicts the actual architecture.	Project migrated from Lovable/Supabase to Spring Boot; README not rewritten.	Replace README.md with README-SETUP.md content plus an architecture summary.
Default JWT secret fallback could silently secure production.	application.yml provides a default expression for JWT_SECRET.	Refuse to start when the secret is missing or equals the documented default.
ESLint gate fails (100 errors / 28 warnings).	Generated/adapted code never passed a lint gate; e.g. require() import in tailwind.config.ts.	Fix auto-fixable issues, tune rules, add lint to CI.
1.9 MB single JS chunk slows first load.	No route-level code splitting; heavy libs (xlsx, jspdf, html2canvas, recharts) eagerly bundled.	React.lazy per route and manualChunks for vendor libraries.
HR/leave/timesheet listing paths can load unbounded data.	visibleRequests/visibleSheets use findAll().limit(200); team attendance lacks pagination.	Keyset/page queries with filters and total counts.
Timesheet bulk approve swallows exceptions silently.	bulkApprove catches and ignores per-id exceptions, returning only a count.	Collect per-item results/errors and report them in the response.
Approver resolution heuristic can notify the wrong people.	HrmsEventConsumer.isApprover treats top-level profiles (manager == null) as approvers.	Resolve approvers from profile.manager or role assignments.
Dual role systems increase cognitive load.	Legacy user_roles kept alongside dynamic RBAC for Supabase parity.	Migrate fully to RBAC assignments; keep the legacy column read-only for one release, then drop.
Leave cancellation deletes the row (no trail).	cancel() deletes the entity rather than marking it cancelled.	Soft-delete with a CANCELLED status (consistent with the other state machines).
Change-password does not revoke other sessions.	changePassword only re-encodes the password.	Call revokeAllSessions after a successful change.
Audit tables have no retention policy.	Only outbox rows are purged (7 days).	Scheduled archival/partitioning for audit tables.
frontend/.env committed (harmless: VITE_API_BASE_URL only) and .tsbuildinfo artifacts present.	Generator defaults.	Add .env and *.tsbuildinfo to .gitignore.

27 Current Strengths

- Sound architectural choices:
  transactional outbox, Redis idempotency, Kafka DLT, stateless JWT with rotation - patterns usually seen in production systems, applied consistently.
- Clean code organisation:
  package-by-feature with controller/service/repository separation; DTO records colocated with controllers; central exception model; Swagger documented endpoints.
- Security depth:
  hashed refresh tokens, rotation, lockout, rate limiting, permission checks at every boundary, no committed secrets.
- Operational readiness:
  one-command Docker environment, Prometheus/Grafana pre-provisioned, correlation-ID logging, health endpoints, rolling logs.
- Schema discipline:
  Flyway migrations with constraints as backstops and ddl-auto validate to catch drift.
- Modern frontend engineering:
  TypeScript throughout, TanStack Query, central API client with refresh single-flight, 48-component accessible UI kit.
- Documentation trail:
  README-SETUP.md is accurate and includes a 10-point manual verification checklist matching the actual features.

28 Known Limitations

Severity	Limitation
High	No CI/CD pipeline and no automated lint/test gates; correctness currently relies on manual runs.
High	JWT secret fallback permits a misconfigured production deployment (Section 19).
High	Frontend test coverage is zero; backend coverage limited to auth/token/migration areas (10 tests).
Medium	Some HR listing endpoints are unbounded; scale risk with data growth (Section 23).
Medium	Approver fan-out heuristic may email the wrong set of approvers.
Medium	WebSocket auth uses a query-parameter token; tokens live in localStorage.
Medium	Legacy dual role system maintained in parallel with RBAC.
Medium	No code-splitting; 1.9 MB initial bundle.
Low	README.md is outdated boilerplate; no LICENSE file; no CHANGELOG.
Low	Supabase edge functions and migrations are dead weight for the current stack (kept as history).

29 Recommended Improvements

High priority

- Add CI/CD (GitHub Actions):
  mvn verify + dockerised Testcontainers tests, npm lint/build, dependency scanning. Why: the project has good tests but no automated gate; regressions are currently invisible.
- Remove the JWT secret fallback:
  fail fast on missing/default secret. Why: prevents an silently insecure production deployment.
- Fix the lint baseline:
  clear the 100 errors. Why: restores a meaningful static-analysis gate.
- Harden proxy trust for rate limiting:
  only honour X-Forwarded-For from known proxies. Why: the current limiter is bypassable.
- Add frontend tests:
  Vitest + React Testing Library for api.ts refresh logic, useAuth and one approval workflow; consider a Playwright smoke test. Why: auth and money-adjacent flows deserve regression cover.

Medium priority

- Paginate/filter all HR listing endpoints; add manager-scoped team attendance filtering.
- Introduce route-level code splitting and vendor chunking.
- Move approver resolution to explicit manager/role relationships.
- Adopt httpOnly cookie refresh tokens or at minimum move WebSocket auth off the query string.
- Add security headers (HSTS/CSP/X-Content-Type-Options) at nginx.
- Document JWT/secret rotation and add audit-table retention.

Low priority

- Replace README.md; add LICENSE, CHANGELOG and PR templates.
- Remove/retire Supabase artifacts once the migration is signed off.
- Add OpenTelemetry tracing; expose actuator/log correlation in Grafana panels.
- Add per-user rate limits and request body size limits for bulk endpoints.

30 Future Enhancements

- Shift scheduling:
  rosters, shift swaps and policy evaluation per shift - the policy table and grace-period logic extend naturally.
- Payroll export:
  approved timesheets + attendance already provide the inputs for period payroll extracts (client-side PDF/XLSX tooling exists).
- Native calendar integrations:
  extend the CalendarImport concept to Google/Outlook sync for automatic timesheet drafts.
- SSO expansion:
  SAML/Okta/Azure AD via Spring Security, building on the existing OAuth2 configuration.
- Two-factor authentication:
  TOTP second factor reusing the one_time_tokens mechanism.
- Notification preferences:
  per-user channel and digest settings; email templates already centralised in EmailTemplates.
- Document storage:
  employee document management (object storage + metadata table).
- Advanced analytics:
  trend dashboards over audit/attendance history; anomaly hints (chronic lateness, unused balances).
- Mobile/PWA client:
  the API is fully token-based; a PWA shell would cover clock-in/out on shared devices.

31 Maintenance and Support

Area	Guidance
Environment variables	Everything is configured via .env (names in Appendix). Rotate JWT_SECRET periodically; note that rotation invalidates all sessions.
Database maintenance	pgdata is a named compose volume - back it up with pg_dump; Flyway migrations must stay additive; monitor audit table growth (no retention yet).
Kafka / outbox	hrms.events.dlt should be monitored; processed outbox rows are auto-purged after 7 days; kafka-ui (:8085) aids inspection.
Redis	Used for rate limiting, permission cache (5 min TTL) and event dedup (7 d TTL); a flush clears caches safely (limits reset).
Logs	logs/hrms.log rolls daily, 14-day retention; correlationId ties a request across lines; email failures and DLT deliveries are logged at WARN/ERROR.
Email provider	MAIL_ENABLED=false switches to log-only mode; monitor hrms.email.failed metric for delivery issues.
Upgrades	Bump Spring Boot/Boot-managed deps via pom parent; frontend via npm; run browserslist update (data was 15 months stale at build time).
Troubleshooting quick table	429 responses -> rate limit reached; 401 loops -> clear localStorage and re-login (refresh may be revoked); emails missing -> MAIL_ENABLED/MAIL_FROM config; Kafka consumer idle -> check broker :9092 and topic hrms.events; schema error at boot -> entity/migration drift (ddl-auto validate).

32 GitHub / Repository Readiness

Item	State	Assessment
.gitignore	Covers node_modules, dist, .env, backend/target, backend/logs, IDE files	Good
.env.example	Complete variable list with safe placeholders	Good
README.md	Lovable boilerplate; does not describe this stack	Poor (use README-SETUP.md)
README-SETUP.md	Accurate, actionable, includes verification checklist	Strong
Dependency files	package-lock.json committed (frontend); pom.xml pins parent/versions	Good (reproducible)
Docker artifacts	Dockerfiles + compose + monitoring provisioning committed	Strong
Tests	Backend tests present; frontend none; no CI wiring	**Partial**
Secrets	No real credentials committed; compose defaults are dev-only placeholders	Good
Generated files in repo	frontend/.env (benign), *.tsbuildinfo, node_modules present locally (ignored)	Minor cleanup
Documentation of legacy	Supabase artifacts kept with explanatory comments	Acceptable

Overall readiness: Medium-High. With a rewritten README, a LICENSE, and a CI workflow running lint + tests, the repository would be well above average in reproducibility and onboarding clarity for a project of this size.

33 Resume / Portfolio Value

Skill	Evidence in This Project
Full-stack development	End-to-end feature delivery: schema, API, business rules, UI (e.g. the timesheet lifecycle spans V1 schema -> TimesheetService -> Timesheets page).
Java 21 / Spring Boot 3	Modern Boot 3.3 idioms: records, validation, @ConfigurationProperties, security filter chains, scheduling.
REST API design	56 route groups, versioned paths, DTO-per-controller, OpenAPI/Swagger documentation.
Relational database design	22-table schema with FKs, CHECK/UNIQUE constraints, composite indexes, Flyway migrations + seed.
Authentication & authorization	JWT with rotation, OAuth2 (Google/GitHub), BCrypt, lockout, dynamic RBAC engine.
Event-driven architecture	Transactional outbox, Kafka pub/sub, DLT handling, Redis idempotency.
Caching & rate limiting	Redis-backed permission cache with invalidation and sliding-window throttles.
Real-time systems	WebSocket notification push with reconnection strategy.
React / TypeScript UI engineering	Vite SPA, TanStack Query, hooks architecture, shadcn/ui design system, dark theme.
Containerization & DevOps	Multi-stage Dockerfiles, 8-service compose with healthchecks, Prometheus/Grafana provisioning.
Observability	Micrometer business metrics, correlation-ID logging, structured log patterns.
Testing	Unit (Mockito) + integration (Testcontainers PostgreSQL) suites wired into Maven.
Security engineering	Token hashing, no user enumeration, error hygiene, CORS allow-listing, secrets hygiene.

34 Final Technical Summary

Dimension	Summary
Architecture	Modular monolith (Spring Boot 3.3, Java 21) + React 18 SPA; event-driven notifications via transactional outbox + Kafka; Redis for cache/rate-limit/dedup; PostgreSQL 16 with Flyway.
Technology	Java 21, Spring Boot 3.3.5, JJWT 0.12.6, Redis 7, Kafka 3.7, React 18.3, TypeScript 5.8, Vite 5.4, Tailwind 3.4, shadcn/ui, Docker, Nginx, Prometheus, Grafana.
Features	Auth (local + OAuth2), attendance with corrections/policies/holidays, leave governance, weekly timesheets with bulk approvals, employee directory, RBAC, performance reports, in-app + email notifications, audit, exports.
API surface	56 REST route groups across 8 controllers + WebSocket channel; Swagger UI included.
Data flow	Validated request -> transactional service (RBAC + rules + audit) -> PostgreSQL; side-effects via outbox -> Kafka -> in-app/WS + email with retries and DLT.
Security	Strong baseline: BCrypt 12, hashed rotating refresh tokens, lockout, rate limits, endpoint-level RBAC, no committed secrets. Hardening items identified in Section 19.
Testing	10 backend tests (unit + Testcontainers integration) wired into Maven; no frontend tests; suite not executable in the authoring environment.
Verification (this report)	Frontend build PASS (8.63 s, chunk warning); lint FAIL (128 problems); schema/config/API/secret reviews PASS; backend and stack runtime checks not executable in this environment - honestly marked.
Performance	Indexes, Redis caching and async offloading in place; flagged hot spots: unbounded HR listings, approver fan-out, 1.9 MB bundle.
Deployment	One-command 8-service Docker Compose stack with healthchecks and pre-provisioned monitoring; no CI/CD or Kubernetes yet.
Current status	Functionally complete core; production-readiness gated by CI, hardening items and data-volume fixes prioritised in Sections 26/29.

35 Appendix

A. Environment Variables (names only - values never appear in this report)

DATABASE_URL, DATABASE_USERNAME, DATABASE_PASSWORD, JWT_SECRET, JWT_ACCESS_EXPIRATION, JWT_REFRESH_EXPIRATION, REDIS_HOST, REDIS_PORT, KAFKA_BOOTSTRAP_SERVERS, MAIL_ENABLED, MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM, MAIL_FROM_NAME, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI, GITHUB_CLIENT_ID, GITHUB_CLIENT_SECRET, GITHUB_REDIRECT_URI, FRONTEND_URL, CORS_ALLOWED_ORIGINS, GRAFANA_PASSWORD (compose), VITE_API_BASE_URL (frontend).

B. Seeded Permission Matrix (V2__seed.sql)

Module	Actions	Super Admin	HR Admin	Manager	Employee
attendance	view, create, edit, delete, approve	All	view/create/edit/approve	view/edit	view/create
timesheets	view, create, edit, delete, approve	All	view/create/edit/approve	view/approve	view/create/edit
leave_management	view, create, edit, delete, approve	All	view/create/edit/approve	view/approve	view/create/edit
role_management	view, create, edit, delete, assign	All	-	-	-

C. Seeded Reference Data

- Roles: Super Admin, HR Admin, Manager, Employee (is_system_role = true).
- Leave types: Annual (25 d, carry-forward), Sick (15 d), Personal (5 d), Emergency (3 d).
- Attendance policy: 'Default Policy' 09:00-17:00, 60-min break, 15-min grace, 8 h overtime threshold.

D. Glossary

Term	Definition
Outbox pattern	Persisting events in the same DB transaction as business data, then publishing asynchronously - no lost or phantom events.
DLT (dead-letter topic)	Kafka topic receiving messages that repeatedly fail processing, for later inspection.
Refresh token rotation	Each refresh issues a new refresh token and revokes the previous one, limiting replay of stolen tokens.
RBAC	Role-based access control: users -> roles -> permissions (module:action) checked by services.
Effective role	Merged role derived from the legacy user_roles enum and dynamic RBAC assignments.
MDC	Mapped Diagnostic Context - per-request log fields (correlationId, userId) in Logback.
Hikari	JDBC connection pool used by Spring Boot (max 10 connections here).
Testcontainers	JUnit library running throwaway Docker containers (real PostgreSQL) for integration tests.

E. Verification Information

- Report generated: September 4, 2026, from the workspace copy of slate-timesync.
- Executed: npm run build (PASS), npm run lint (FAIL, 128 problems), environment checks (node v24.19.0, npm 11.17.0; Java/Maven/Docker absent).
- Static reviews: schema (V1/V2), configuration consistency, API mapping scan (81 mapping annotations / 56 route groups), secrets scan, dependency inventory.
- Not executed (marked throughout): mvn test / mvn package, docker compose startup, runtime API smoke tests.